

按一下按鈕，電腦怎麼知道？

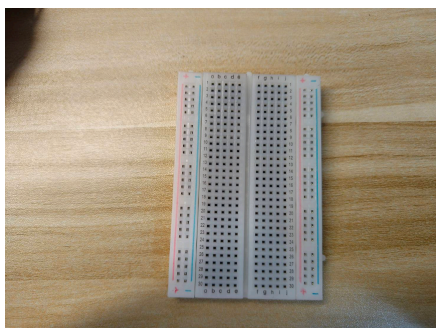
先看懂線怎麼接通，再看程式怎麼讀。

這段完成後，你要能解釋：

按鈕改變哪一條路？GPIO 為什麼讀到不同狀態？電腦上的紀錄從哪裡來？

同一件事，分成四步看

- 1 一條線 用電表確認兩端是否相通。
- 2 麵包板 找出原本就在板內接通的孔。
- 3 按鈕 比較放開和按下時，哪些腳接通。
- 4 ESP32 程式 把 GPIO 讀值解釋成按下或放開，再送出紀錄。



先拿這些就好

電表、麵包板、杜邦線、四腳按鈕。

照片用來認實物；後面的孔位圖才用來找位置。

先不接電源。接下來三頁的通斷練習，不接 ESP32、USB、電池或其他電源。電表會用自己的電池進行小電流測試。

下一頁：先讓電表自己證明，它能分辨「接觸」和「分開」。

電表先回答：通，還是不通？

這次只用通斷功能，不量電流，也不量供電電壓。



實物照片的旋鈕在 OFF。
接好表筆後，還要選通斷檔。

先認三個位置

1. 黑筆插 COM。
在這台表的下方中間。
2. 紅筆插 VΩmA。
在下方右邊。左邊 10A 不用。
3. 旋鈕轉到通斷蜂鳴符號。
找聲波圖案，不是 200 Ω 的數字。實物標示不同時，先一起核對，不猜檔位。

先做自測，再測一條線

你的動作	預期	我的實際結果
兩個筆尖分開	不叫	_____
兩個金屬筆尖接觸	會叫	_____
同一條公對公線的兩端，各碰一筆	完好的線會叫	_____

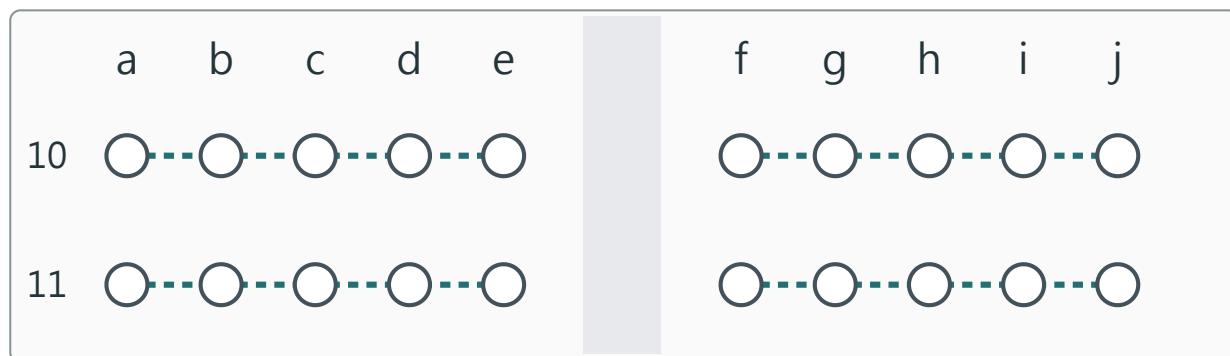
怎麼判讀：會叫，表示兩筆間有足夠低電阻的路徑；不代表每次都是故障。這裡是刻意讓筆尖接通。

結果不符時，停在這頁，檢查插孔、檔位與接觸。不要改接電源來試。通斷檔不能用在帶電電路。

下一頁：用同一個方法，找出麵包板內部的連接。

麵包板：哪些孔原本就相通？

孔靠得很近，不一定相通。用電表驗證，不只看外觀。



灰色中央槽把左右兩組分開；虛線表示板內連接。

示意圖只畫中央接線區兩列，不含兩側電源軌；依實物字母與列號找孔。

先讀一個孔位

a10 是 a 欄、第 10 列。這張圖中，a10 到 e10 是同一組；f10 到 j10 是另一組。

用兩條杜邦線把測試位置引出來

1. 在要比較的兩個孔，各插入一條公對公杜邦線。
2. 紅、黑表筆各碰一條線另一端的金屬針。
3. 兩條線末端不要互碰；不要把粗表筆硬塞進麵包板孔。

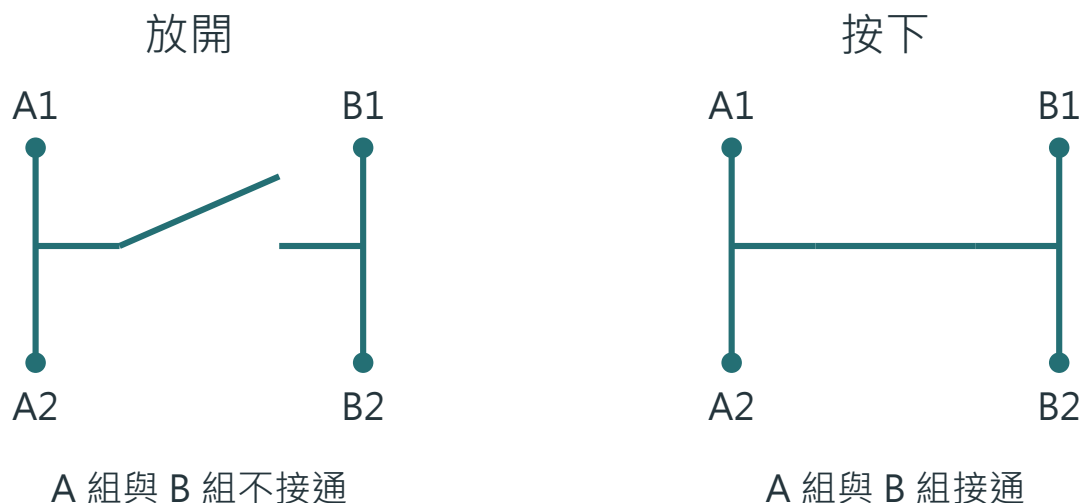
比較的兩孔	先猜：叫 / 不叫	實測：叫 / 不叫
a10 與 e10	_____	_____
a10 與 a11	_____	_____
a10 與 f10	_____	_____

核對：正常、未加其他接線的這類麵包板，三次應是「叫、不叫、不叫」。不符時先確認列號、線是否插到底，以及表筆接觸。

下一頁：麵包板的連接不會自己變；按鈕則會改變連接。

按鈕不是四個互不相干的腳

先找「一直相通」的一組，再找「按下才相通」的兩組。



這是內部連接示意，不是腳位擺放圖。A1、A2、B1、B2 是本頁為說明測試而標的腳，必須實測辨認。

按鈕先不要裝進其他電路

1. 放開按鈕，用通斷檔比較各腳，找出原本就相通的兩腳，記為 A1、A2。
2. 確認另兩腳放開時也相通，記為 B1、B2。
3. 紅筆碰 A1、黑筆碰 B1。固定接觸後，再比較放開與按下。

量哪兩腳	放開時的預期	按下時的預期
A1 與 A2 (同組)	會叫	會叫
A1 與 B1 (跨組)	不叫	會叫

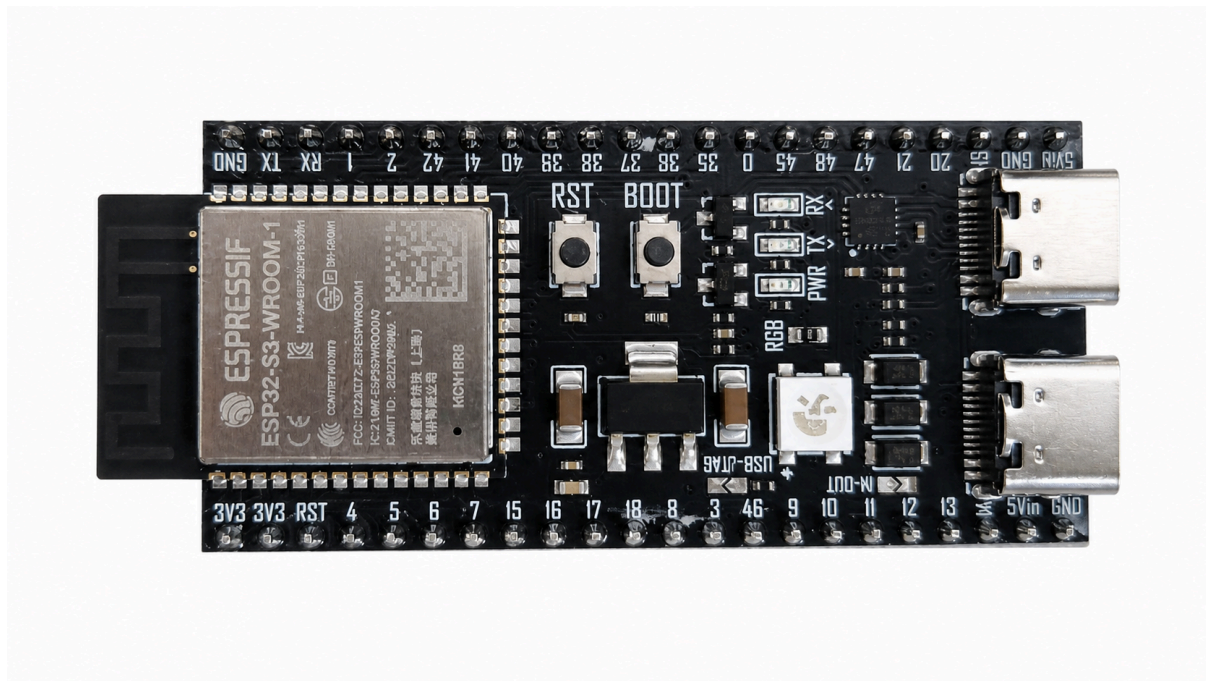
最重要的判斷：要讓按鈕改變連通，必須從兩組各取一腳。只取同組兩腳，按不按都相通。

我的按鈕哪兩腳是同組？在紙上畫四個腳，把相通的一組圈起來。若測不出上表結果，先一起核對接觸與腳位，不接 ESP32。

下一頁：拿出開發板，先找到要接線的兩個腳位。原理等看到紀錄後再講。

拿出開發板，找到兩個腳位

先找印字，不接線。這一段只用 GPIO4 與 GND。



照片：課堂既有 YD-ESP32-S3 Type-A V1.5。把你的板子轉成相同方向：天線在左，兩個 USB 接頭在右。

跟著找到，先用手指出來

這次找哪個腳	照片中的位置
印字「4」	下排，印字 RST 的右邊、5 的左邊。這是 GPIO4，不是「數第四個腳」。
印字「GND」	上排最左端，靠天線這一側。這次統一使用這個 GND。

板子先放桌上：放在乾燥、不導電的平面，不插進這塊 400 孔麵包板。稍後用公對母杜邦線連過去。

本接法只對應圖示板型。你的印字或板型不一樣，就停在辨認這一步，和全班一起核對，不靠外形猜腳位。板上若還有其他作品的接線，先斷開所有電源，另用空白器材準備本實驗。

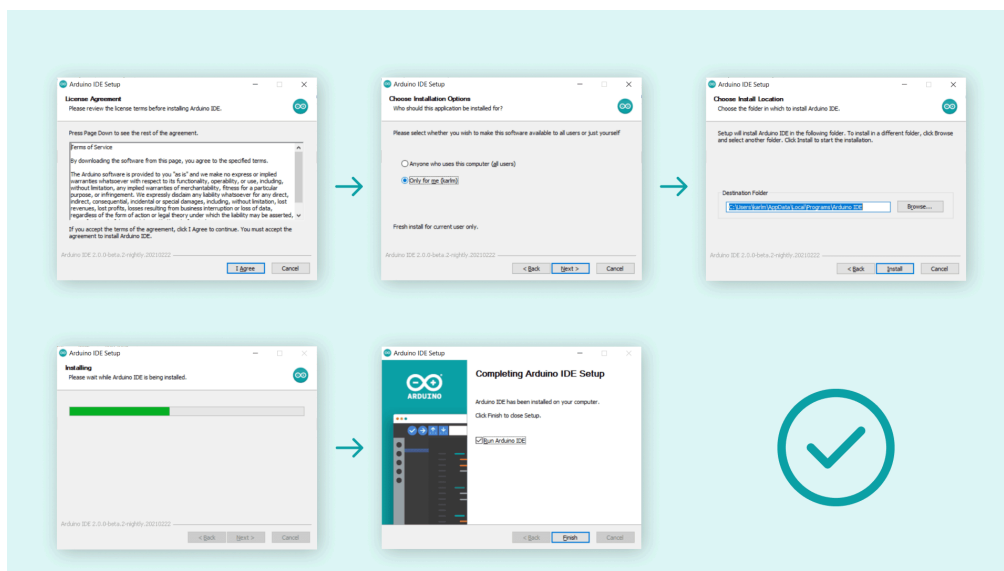
下一頁：從安裝電腦軟體開始，先讓 ESP32 傳回一句 Hello。還不接按鈕。

先安裝 Arduino IDE

它是電腦上的軟體：我們用它寫程式，再把程式放進 ESP32。

以下以 **Windows 11 / Arduino IDE 2** 操作。板子先不用插 USB，也不接杜邦線；這一步只操作電腦。

1. 打開瀏覽器，進入 [Arduino 官方下載頁](#)。
2. 找 **Arduino IDE 2**，選 Windows 64-bit 的 **.exe 安裝檔**。不要選網頁版 Cloud Editor。
3. 下載完成後，開啟電腦的「下載」資料夾，連按兩下剛下載的 .exe。
4. 閱讀授權後，願意接受才按 **I Agree**。個人電腦可選 **Only for me**，再按 Next、Install。
5. 等安裝完成，勾選 **Run Arduino IDE**，按 Finish。已安裝 IDE 2 的同學直接從開始選單打開。



Arduino 官方安裝流程總覽，使用早期 IDE 2 畫面。實際版本號、使用者名稱與路徑不必相同；以目前安裝程式文字為準。

做完應看到：Arduino IDE 的程式編輯視窗。第一次開啟可能還在下載必要工具，等它完成。

學校電腦若沒有安裝權限，先一起處理，不繞過管理限制。本講義不要求雲端帳號，也不需要付費方案。

下一頁：在軟體裡加入 ESP32 的安裝來源。

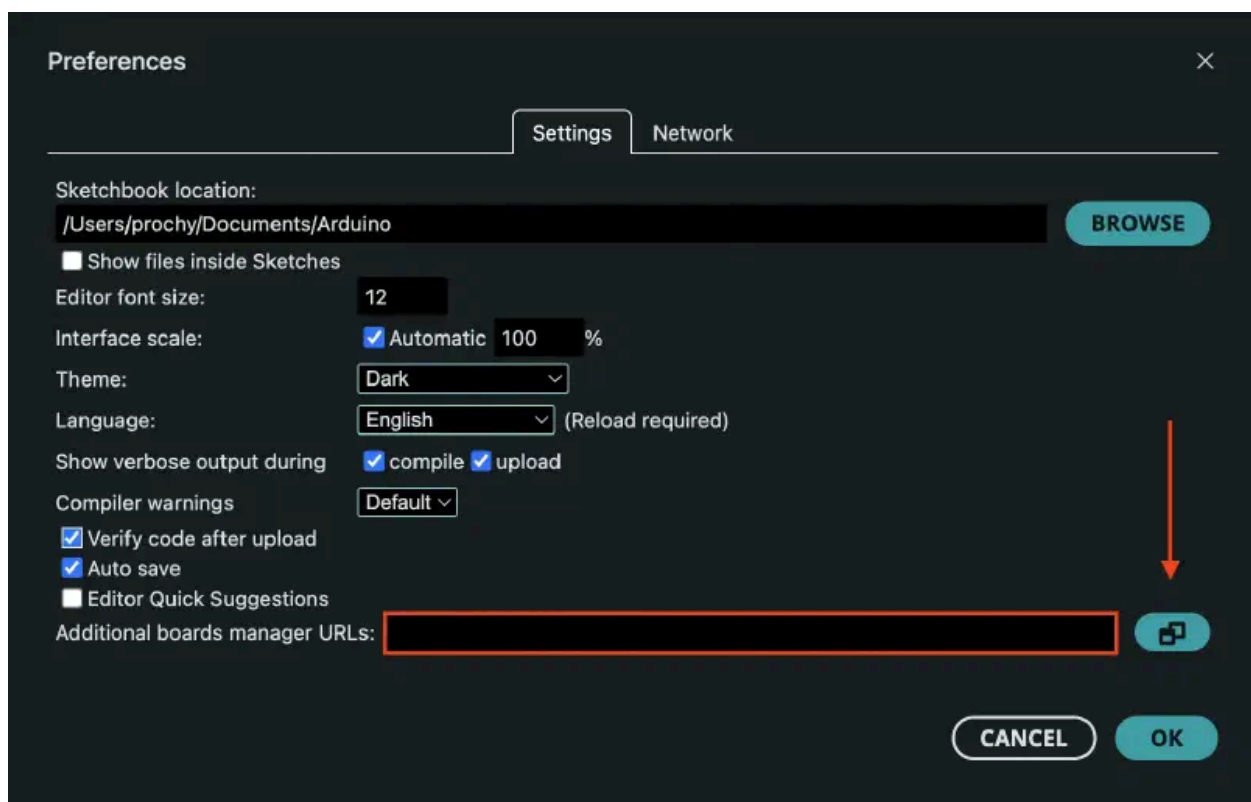
操作依據與畫面來源：[Arduino 安裝說明](#)。圖片未修改；版本差異見圖說。

告訴軟體去哪裡找 ESP32 支援

剛裝好的 Arduino IDE，還需要知道如何處理我們這種板子。

1. 在視窗上方點 **File (檔案) → Preferences (偏好設定)**。
2. 找到 **Additional boards manager URLs** 欄位。可以點右側按鈕開啟清單。
3. 貼上下面這個網址。若原本已有其他網址，保留它，另加一行；不要覆蓋別人的設定。

https://espressif.github.io/arduino-esp32/package_esp32_index.json



Espressif 官方偏好設定畫面。只找框出的 Additional boards manager URLs；不要照抄圖中個人路徑、主題或其他設定。

按 **OK** 關閉網址清單，再按偏好設定視窗的 **OK**。

做完應確認：重新開啟 Preferences，剛貼上的網址還在。網址中是 `package_esp32_index.json`，沒有 `dev`。

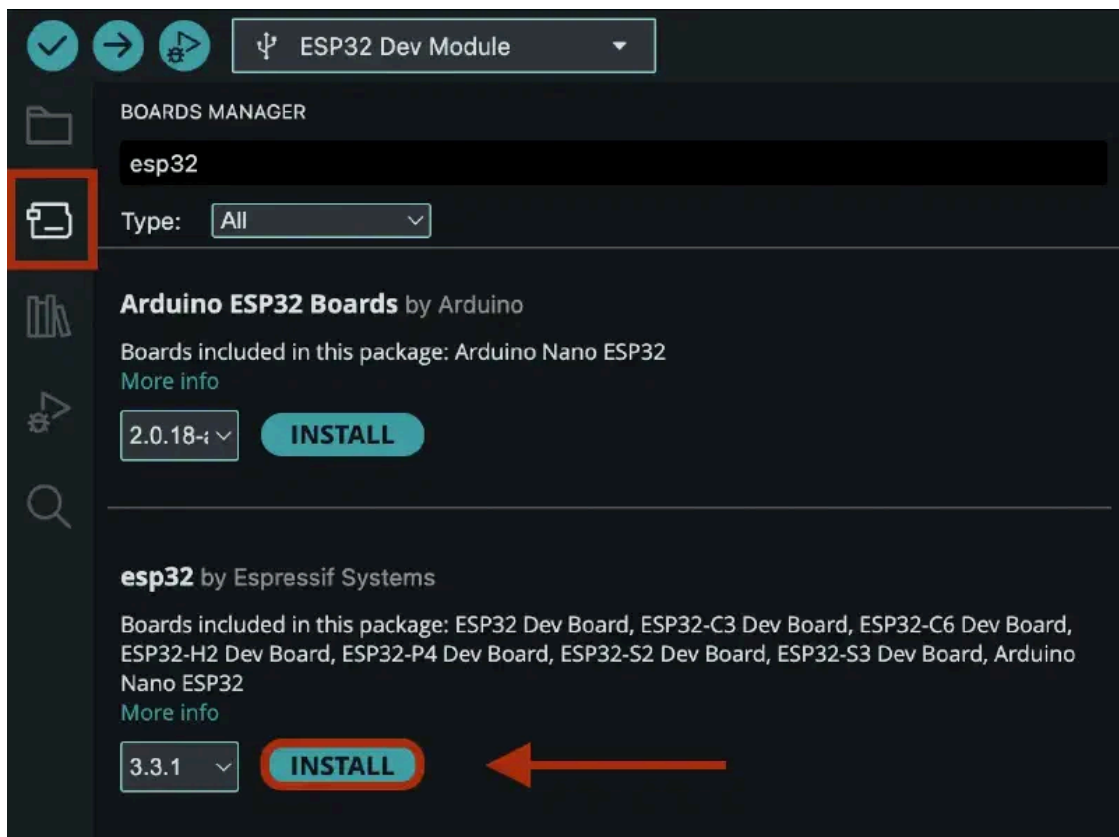
下一頁：下載並安裝 ESP32 的支援工具。

操作依據與畫面來源：[Espressif 安裝教學](#)。圖片未修改；版本差異見圖說。

找到 esp32 by Espressif Systems

這一步把處理 ESP32 程式需要的工具裝進電腦。

1. 點上方 **Tools (工具)** → **Board (開發板)** → **Boards Manager (開發板管理員)**。
2. 在搜尋欄輸入 **esp32**。找到作者是 **Espressif Systems** 的那一項。
3. 打開該項的版本選單，選 **3.3.11**，按 **INSTALL**，等下載與安裝完成。



Espressif 官方管理員畫面：本課要下面的 esp32 by Espressif Systems，不是上面的 Arduino ESP32 Boards。

不要照抄圖片的版本與板名。圖中是舊版 3.3.1；本課選 3.3.11。上方 ESP32 Dev Module 也不是本課稍後要選的 ESP32S3 Dev Module。

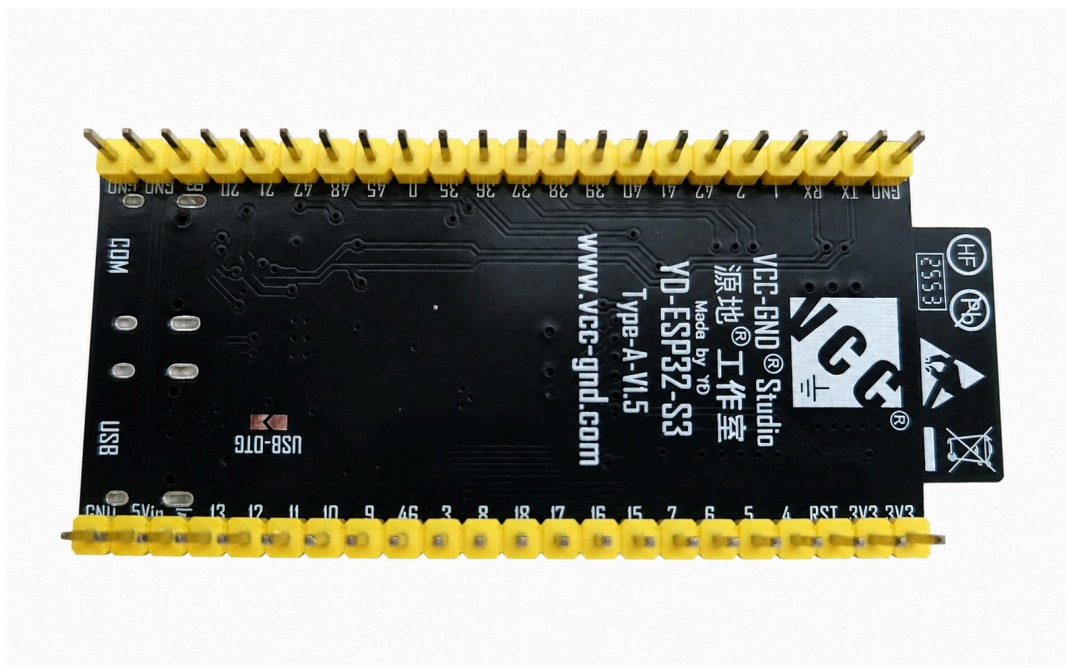
做完應看到：這一項顯示 **3.3.11 installed** 或已安裝狀態。先確認不是仍在下載，再關閉並重新打開 Arduino IDE。

下一頁：現在才把開發板接到電腦。

操作依據與畫面來源：[Espressif 開發板管理員教學](#)。圖片未修改；版本差異見圖說。

把 ESP32 接到電腦

使用可以傳資料的 USB 線；有些線只能充電，不能傳程式。



這是課堂板子的背面照片。本頁照片的 USB 接頭在左側；找印字 COM 對應的接頭，不靠正反面翻轉後的上下位置猜。

1. 板子沒有杜邦線、按鈕、電池或其他模組。把板子翻過來，找到印字 **COM** 的接頭。
2. 先看電腦軟體的 **Tools → Port** 清單。Port 是電腦用來識別連接裝置的通訊埠；現在沒有清單也沒關係。
3. 把 USB 線一端插電腦，另一端插板子的 **COM** 接頭。先只接這一塊板。
4. 把板子正面朝上放回乾燥、不導電的桌面，等 Windows 辨識，再重開 **Tools → Port**。

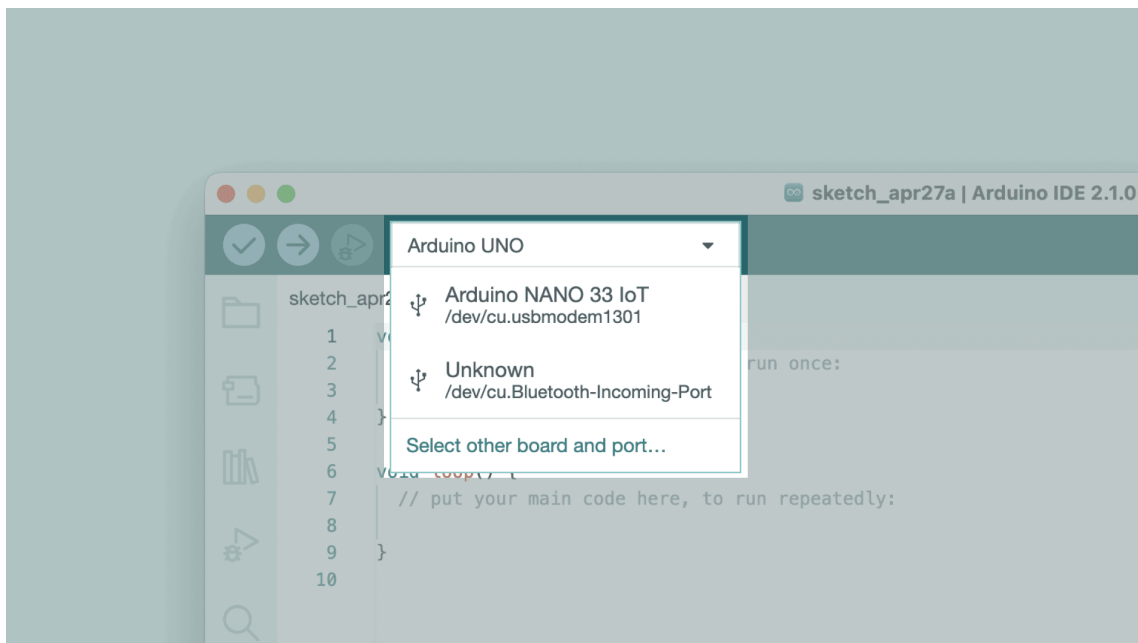
記下新增的名稱：COM_____。例如 COM8 只是某次電腦分配的號碼，不是全班都選 COM8。

板背的 **COM** 是接頭標示；電腦上的 **COM8** 之類是軟體名稱。PWR 燈亮只表示板上有供電，還不能證明資料線或程式傳輸正常。

下一頁：選板型與剛才新增的 Port。若沒有新增 Port，先看第 33 頁，不先亂選。

選 ESP32S3 Dev Module

這個選單告訴軟體「程式要給哪一種板子」。另一個 Port 選單則指定連哪一塊。



Arduino 官方選單截圖，為 macOS / 其他板型示例。只看下方 Select other board and port 的入口；不選圖中的 UNO、Nano 或 Bluetooth。

1. 點頂端的開發板下拉選單，再點 **Select other board and port...**（選擇其他開發板與連接埠）。
2. 在 **Boards** 搜尋欄輸入 **ESP32S3 Dev Module**，點選完全相同的名稱。留意中間有 **S3**。
3. 在 **Ports** 清單，選第 9 頁剛記下的 COM 號碼，再按 **OK**。

你現在選的是	應該是什麼
Board (板型)	ESP32S3 Dev Module
Port (連接埠)	我的 COM_____，不是老師的固定號碼

清單若只寫 **Unknown**，不代表板子壞了；仍可手動選板型。若完全找不到 ESP32S3 Dev Module，回第 8 頁檢查安裝是否完成。

下一頁：跟著選好這塊 N16R8 板子需要的選項。

操作依據與畫面來源：[Arduino 板型與 Port 說明](#)；圖見 [Arduino 上傳教學](#)。圖片未修改；版本差異見圖說。

在 Tools 選單照表選一次

設定就是在選單裡選項目。這頁先照著操作，還不用理解每個英文名稱。

先確認第 10 頁已選 **ESP32S3 Dev Module**，再點上方 **Tools (工具)**。每選完一項，重新打開 Tools 選下一項。

在 Tools 找這個名稱	選右邊這個值
Upload Speed	115200
USB Mode	Hardware CDC and JTAG
USB CDC On Boot	Disabled
Upload Mode	UART0 / Hardware CDC
Flash Mode	QIO 80MHz
Flash Size	16MB (128Mb)
Partition Scheme	16M Flash (3MB APP/9.9MB FATFS)
PSRAM	OPI PSRAM
Erase All Flash Before Sketch Upload	Disabled

其他項目先不改。這張表只對應本講義照片中的 **YD-ESP32-S3 Type-A V1.5 / N16R8**，以及 **esp32 3.3.11**。

一起核對：重開 Tools，依序對照表格。找不到選項時，先核對板型和安裝版本，不隨便挑名字相近的項目。

這裡的 115200 是「把程式傳進板子」的速度。稍後訊息視窗也會用到 115200，但那是另一個設定，兩處都要各自選。

下一頁：第一次上傳程式，先讓板子只做一件事：傳回 Hello。

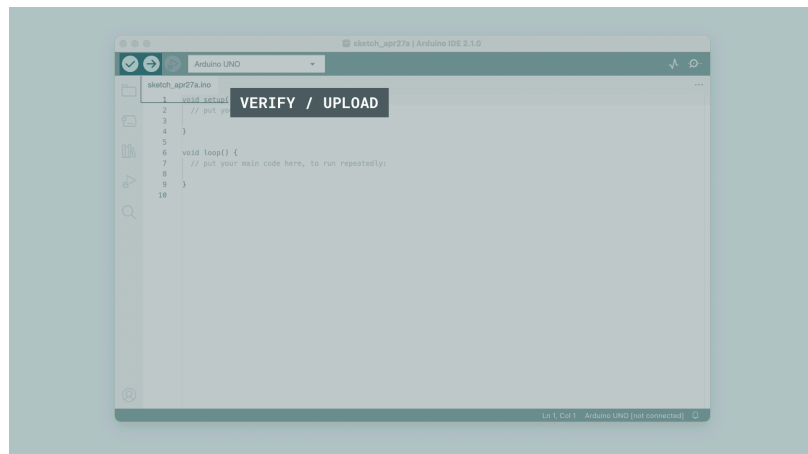
操作依據與畫面來源：[Espressif Tools 選單](#)；選項文字另核對本機 3.3.11。圖片未修改；版本差異見圖說。

把 Hello 程式放進 ESP32

「上傳」就是從電腦經 USB，把程式傳進板子；不是傳到網路。

1. 點 **File** → **New Sketch**（檔案 → 新增草稿），開一份新的程式，依第 10–11 頁核對板型、Port 與選項。
2. 在中間白色或黑色的程式編輯區點一下，按 **Ctrl+A** 全選，整段換成下面內容。
3. 按 **Ctrl+S**，命名為 **hello_first**。仍只接 USB，不接杜邦線。

```
void setup() {  
  Serial.begin(115200);  
}  
  
void loop() {  
  Serial.println("Hello");  
  delay(1000);  
}
```



Arduino 官方工具列截圖。左上方勾勾是 Verify；右箭頭是 Upload。圖中 UNO 不是本課板型，本課仍選 ESP32S3 Dev Module。

按左上方右箭頭 **Upload**，先等程式檢查，再等傳輸；中途不拔 USB。若跳出另存視窗，先完成儲存。

做完應看到：IDE 顯示 **Done uploading** 或上傳完成。只有「編譯成功」不等於已傳進板子。錯誤時看第 33 頁。

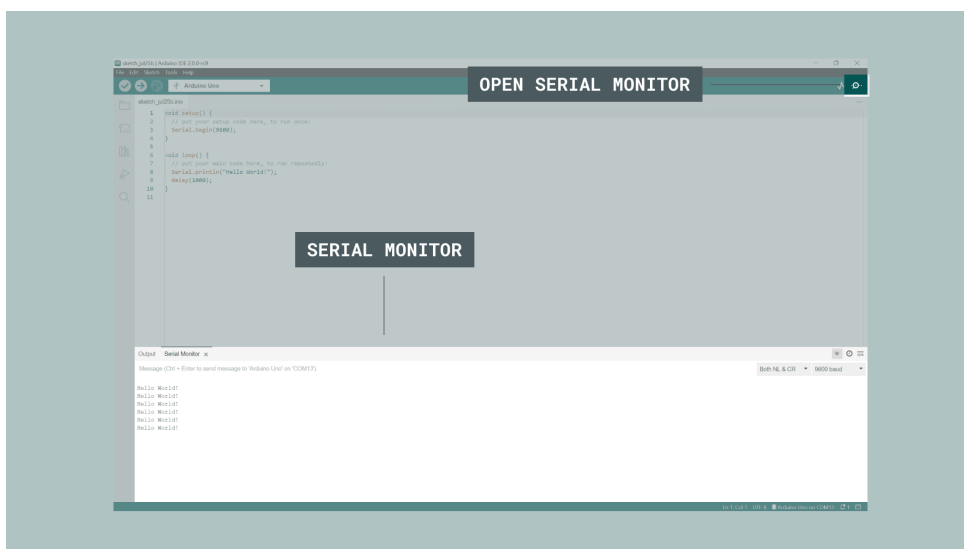
下一頁：打開訊息視窗，看板子有沒有送回 Hello。

操作依據與畫面來源：[Arduino 上傳教學](#)。圖片未修改；版本差異見圖說。

打開序列監控視窗，看 Hello

Serial Monitor (序列監控視窗) 就是顯示板子傳回文字的地方。

1. USB 保持插著。點 **Tools** → **Serial Monitor** (工具 → 序列監控視窗) ；或點右上方對應圖示。
2. 在下方訊息區找到 **baud** 速度選單，選 **115200**。不要照圖片選 9600。
3. 看下方的 **Serial Monitor** 分頁，不是旁邊顯示編譯訊息的 **Output** 分頁。



Arduino 官方示例只用來辨認右上入口與下方訊息區。本課是 ESP32S3、115200、文字 Hello；不是圖中的 UNO / 9600 / Hello World。

預期每隔約一秒多一行

```
Hello  
Hello  
Hello
```

這是程式的預期示例，不是本次實測紀錄。看到連續三行後，先不接按鈕，確認全班都找到訊息區。

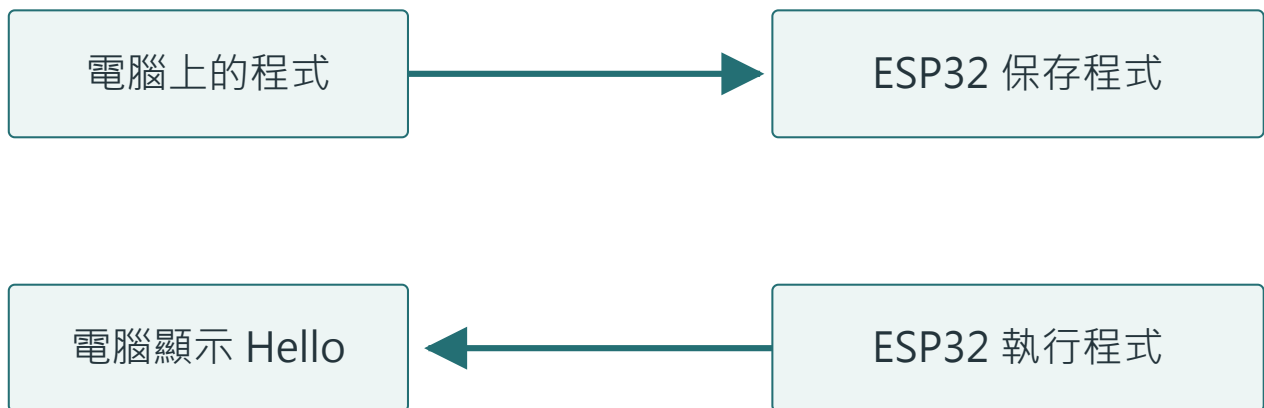
再試一次：關閉 Serial Monitor 分頁，把程式的 "Hello" 改成 "Hi"，儲存並再按 Upload。重新開啟訊息視窗後，應改成 Hi。

下一頁：先用剛才的結果，解釋電腦與板子各做了什麼。

操作依據與畫面來源：[Arduino Serial Monitor 教學](#)。圖片未修改；版本差異見圖說。

剛才 Hello 是誰送出來的？

先分清楚「把程式傳進去」和「把執行結果傳回來」。



上排：按 Upload 後，程式經 USB 傳進板子。下排：板子執行後，把文字經 USB 傳回電腦。這是資訊流，不是電源接線圖。

剛才看到的程式	先用這句話理解
<code>setup()</code>	板子開始執行時，先做一次準備。
<code>Serial.begin(115200)</code>	準備傳文字，速度用 115200；訊息視窗也要選相同值。
<code>loop()</code>	裡面的動作會反覆做。
<code>Serial.println("Hello")</code>	送出 Hello，然後換行。
<code>delay(1000)</code>	等約 1000 毫秒，也就是 1 秒。

先猜，再看：把現在的 Hi 改成 Bye，只儲存、不按 Upload。訊息區會繼續顯示 Hi，還是變成 Bye？觀察後說明理由。

這個小實驗沒有使用 Wi-Fi，也沒有連雲端。Arduino IDE 是電腦上的工具；真正重複送出文字的是 ESP32。

下一頁：現在才換成按鈕程式，然後拔掉 USB，一條線一條線接。

先換程式，再拔 USB 接按鈕

剛才只會傳回 Hello；現在換成能回報「按下 / 放開」的程式。

在剛才的 Arduino IDE 點 **File → New Sketch**，將編輯區全選，換成下面整段程式，再用 Ctrl+S 存成 **button_follow_along**。板子仍只接 USB，沒有杜邦線。

開新草稿，整段貼上並上傳

```
const int BUTTON_PIN = 4;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  int value = digitalRead(BUTTON_PIN);
  if (value == LOW) {
    Serial.println("pressed");
  } else {
    Serial.println("released");
  }
  delay(300);
}
```

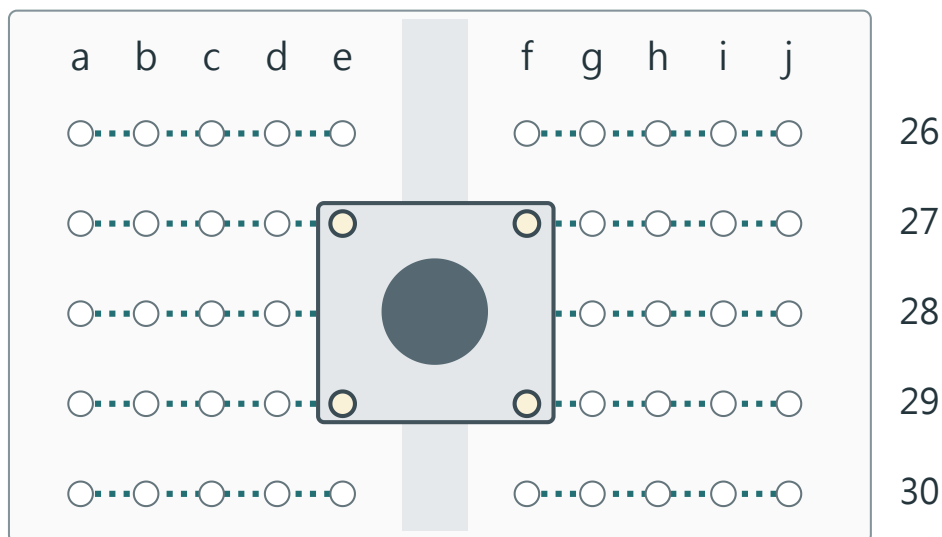
1. 依第 10–11 頁核對板型、Port 與選項；新視窗不一定沿用前一份設定。先關閉 Hello 草稿的訊息視窗。
2. 按左上方右箭頭 **Upload**。完成後點 **Tools → Serial Monitor**，選 115200。
3. 現在還沒有接按鈕，應持續出現 **released**，意思是「放開」。這是預期畫面，不是本次實測紀錄。
4. 看到後，關閉訊息分頁、拔掉 USB，確認 PWR 燈熄滅。再翻下一頁。

未完成上傳或看不到 released，先一起處理程式、Port 與 USB 接頭，不先接線猜原因。此頁不要先理解每行程式，做完按鈕操作後再回來解釋。

下一頁：USB 已拔除，把剛才測過的按鈕放上麵包板。

先放按鈕，暫時不接開發板

拿剛才已測出兩組腳的按鈕，跨中央槽放入。



按鈕四腳：e27、f27、e29、f29

俯視孔位圖。按鈕跨過中央槽；圓點標出四個腳插入的位置。圖中沒有開發板接線。

1. 找到麵包板第 27 列與第 29 列。
2. 將四腳放在 **e27**、**f27**、**e29**、**f29**。若腳距不能自然對孔，不硬壓。
3. 方向要符合：第 27 列兩腳原本同組；第 29 列兩腳是另一組。

用剛才學會的通斷測試一起核對

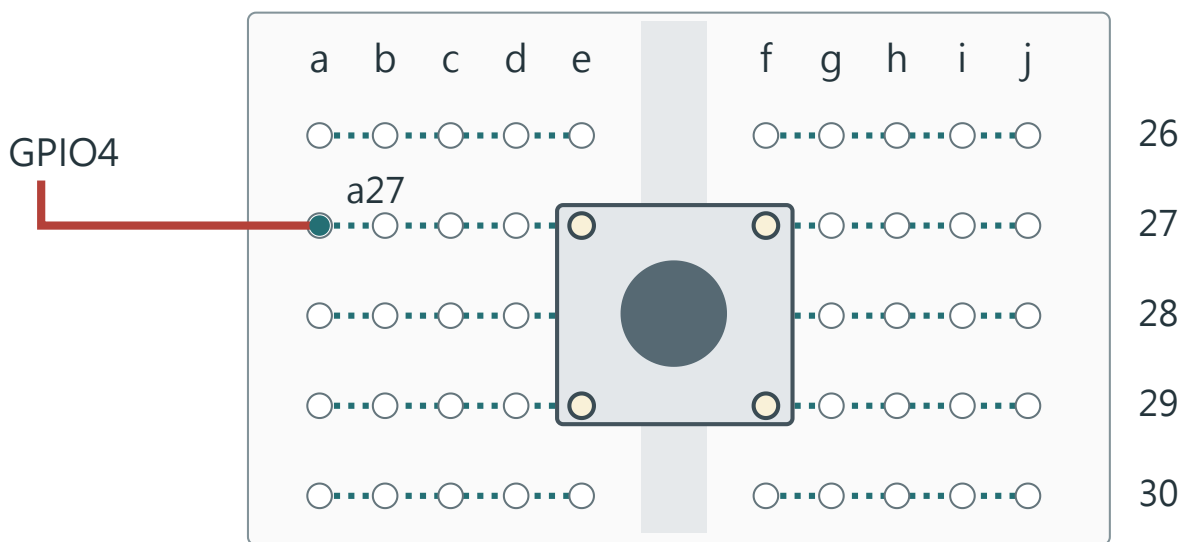
測哪裡	預期
a27 與 j27，按鈕放開	會叫：跨槽兩腳是同組
a29 與 j29，按鈕放開	會叫：另一組也相通
a27 與 a29，先放開，再按住	不叫 → 會叫

沿用兩條公對公線引出測點。結果不同，先找按鈕方向、孔位與接觸；此時不接 USB。完成後移除量測延長線，電表轉 OFF、表筆放旁邊。

下一頁：先接第一條線，從 GPIO4 到 a27。

GPIO4 接到 a27

拿一條公對母杜邦線。線色只是方便辨認，接點才是重點。



按鈕四腳：e27、f27、e29、f29

接線關係圖，不是開發板實體排針排列圖。GPIO4 的實物位置請對照第 5 頁。

這條線只有兩端

哪一端	接哪裡
母頭：有插孔的一端	套到開發板印字「4」的排針
公頭：有金屬針的一端	插麵包板 a27

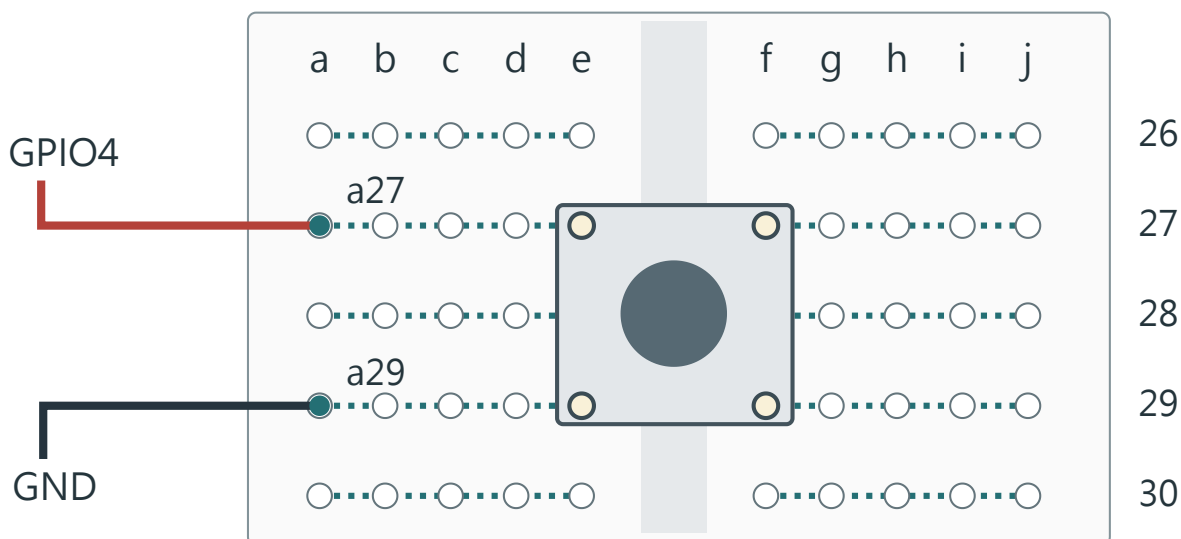
按鈕一腳已在 e27；a27 和 e27 是同一組孔，所以這條線已連到按鈕的一組腳。

一起看目前狀態：板子放桌上，只有這一條線；USB 沒插，3V3、5Vin、GPIO5 都沒有接線。

下一頁：保留第一條線，再接 GND 到 a29。

GND 接到 a29

拿第二條公對母杜邦線。第一條線保持不動。



按鈕四腳：e27、f27、e29、f29

目前完整接法：GPIO4 → a27；GND → a29。兩線分別連到按鈕的兩組，不是接在同一組。

哪一端	接哪裡
母頭	套到第 5 頁指定的 GND 排針
公頭	插麵包板 a29

全班一起照圖核對，現在仍不插 USB

1. 按鈕仍在 e27、f27、e29、f29，方向沒有改變。
2. GPIO4 的線在 a27；GND 的線在 a29。不是 a28，也不是都在第 27 列。
3. 只有這兩條開發板接線；沒有線接 3V3 或 5Vin。金屬針沒有意外碰到旁邊的腳。
4. 電表已 OFF，表筆、量測延長線與其他模組都已移開。

這一段不接 GPIO5 或 TPO，也不用另外接一顆電阻。接點或板型還不確定，就保持斷電，一起對照處理；不靠通電試錯。

下一頁：接法一致後，一起插 USB，看看按鈕能不能改變電腦上的文字。

按下、放開，看文字改變

現在才插回板背標示 COM 的 USB 接頭，不用重新上傳。

1. 打開 Serial Monitor，選同一塊板子的 Port 與 115200。
2. 先不碰按鈕，看見什麼？接著按住約一秒，再放開約一秒。
3. 一起重複三次，對照下表。不要快速點一下就放開。

手的動作	預期會持續出現	我看到的文字
放開	released	_____
按住	pressed	_____
再次放開	released	_____

預期畫面示例，不是實測紀錄

```
released
released
pressed
pressed
released
```

這支程式約每 0.3 秒讀一次並印一行。按住時出現多行 pressed 是這次程式的預期行為，不是「按了很多次」，也不能用來判定按鈕彈跳。

文字不變：先拔 USB，再回第 16–18 頁核對方向與接點。完全沒有文字：先核對 Port、接頭與 115200。改線時不插 USB。

我們現在完成了：手按實體按鈕，電腦上的文字跟著改變。先保留這個結果，接著才解釋它如何發生。

下一頁：回頭看剛才的兩條線，解釋 GPIO 為什麼讀到不同值。

為什麼按下後，文字會改變？

剛才看到 pressed 和 released，現在才拆開其中的原因。

「內部上拉」：板內透過一個電阻，把 GPIO 接點接向 3.3V。按鈕再決定這個接點是否接到 GND。

放開：GPIO 讀到 HIGH



按下：GPIO 讀到 LOW



電氣路徑示意。GND 接回板上電源回路；GPIO 分支是讀取接點狀態，不是把電流送進電腦。

按鈕狀態	接點怎麼了	程式讀值
放開	沒有經按鈕接到 GND	HIGH (高)
按下	經按鈕接到 GND	LOW (低)

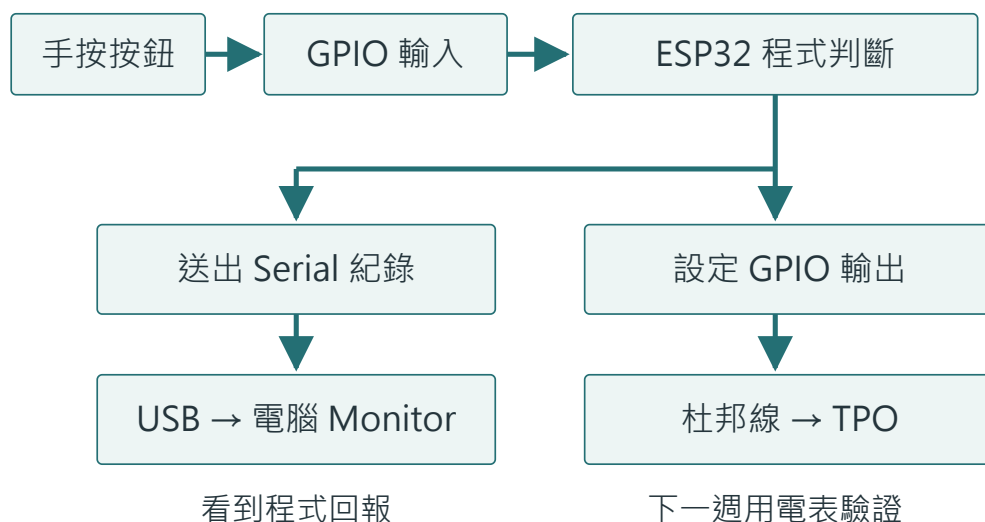
為什麼不是把 3.3V 直接短接 GND？按下時的這條路中間有上拉電阻，它限制電流。不能把電阻省掉，再把 3V3 電源腳直接用線接到 GND。

這個 HIGH / LOW 對應只適用於此圖的上拉接法。不是所有按鈕都「按下 = LOW」。尚未開啟上拉、接錯腳或懸空時，不能套用上表。

回第 15 頁對照：INPUT_PULLUP 開啟上拉；digitalRead 讀狀態；if 把 LOW 對應到 pressed。下一頁分清「讀值、紀錄、輸出」。

紀錄與輸出，是兩條不同的路

左邊是剛才已完成的路；右邊是後續延伸，本次尚未接線。



資訊與控制關係圖。箭頭不是實體接線圖；沒有畫出的供電與 GND 仍須依正式接線圖連接。

先知道為什麼要有 TPO

電腦紀錄告訴我們「程式怎麼回報」。另留一個 GPIO 輸出接點，下一週才能用電表查「腳位電壓是否真的改變」。

舊教材把這個輸出測試位置標成 **TPO**。它不是額外的元件或電源；也不是按鈕資料傳到電腦時必經的位置。

紀錄或動作	能知道什麼	還不能宣稱什麼
程式回報「按下」	程式把讀值判成按下	接線一定全對
程式設定輸出 HIGH	程式要求輸出高電位	已實測到 3.3V

用自己的話回答：剛才沒有接 TPO，為什麼仍能看到 pressed？

在圖上找出「按鈕 → 電腦」的路，再解釋。

先關閉 Monitor、拔掉 USB。下一段加一顆按鈕做計數器，練習一次按壓只計一次；TPO 輸出仍留後續，不與計數器同時接線。

原理核對：[Fluke 通斷測試](#)；[Espressif GPIO](#)。預期結果為教學推演，非本次實測。

兩個按鈕，做一個計數器

沿用剛才的按鈕，讓電腦不只顯示按下，還能顯示累計數字。



資訊流，不是接線圖。數字由 ESP32 計算，經 USB 傳回電腦；本段不用 Wi-Fi、手機或 OLED。

先跟著試這一串動作

從 0 開始 → 加、加、加 → 顯示 3 → 減、減 → 顯示 1。

每次按完都先放開，再按下一次。

這個範例採用的規則	預期結果
按一下加鍵 / 減鍵	加一 / 減一；按住不連續加減
數字是 0，再按減鍵	維持 0，不出現負數
數字是 99，再按加鍵	維持 99；99 是這次選的上限
兩鍵重疊按住	停止接受新計數；兩鍵都放開才恢復
按板上 RST 或斷電重開	重新從 0 開始，不保存上次數字

已有一顆按鈕、兩條公對母線與麵包板。再拿一顆按鈕、一條公對母線、一條公對公線即可。用既有材料，不另外買螢幕。

下一頁：先換成計數程式，再接第二顆按鈕。不要直接沿用舊的 GPIO5 輸出程式。

先讓板子準備好讀兩顆按鈕

這次 GPIO4 和 GPIO5 都讀按鈕；GPIO5 不再用作 TPO 輸出。

1. 關閉 Serial Monitor，拔 USB，移除其他電源。PWR 熄滅後，把開發板上的杜邦線全部拔下；按鈕可留在麵包板上。
2. 開發板單獨放桌上，只接板背標示 **COM** 的 USB 接頭。
3. 有教師的 IoT 資料夾就沿用；否則開課程 [GitHub](#)，按 **Code** → **Download ZIP**，下載後右鍵「全部解壓縮」。
4. 在資料夾依序開 **IOT_Introduction** → **docs** → **teaching_drafts** → **week2_redesign** → **counter_two_buttons**。IDE 選 **檔案** → **開啟**，開裡面的 **counter_two_buttons.ino**，不用抄第 29、30 頁。
5. 先選 **檔案** → **另存新檔**，存成自己的 **counter_practice**。依第 10–11 頁選板型、Port 與選項，按 **Upload** 等上傳完成。
6. 開 **Tools** → **Serial Monitor**，選 **115200**。不按 BOOT，短按一次板上 **RST**，應看到下面這一行。

```
event=start count=0
```

start 表示程式剛開始；**count=0** 表示目前數字是 0。這是預期文字，不是本次硬體實測。

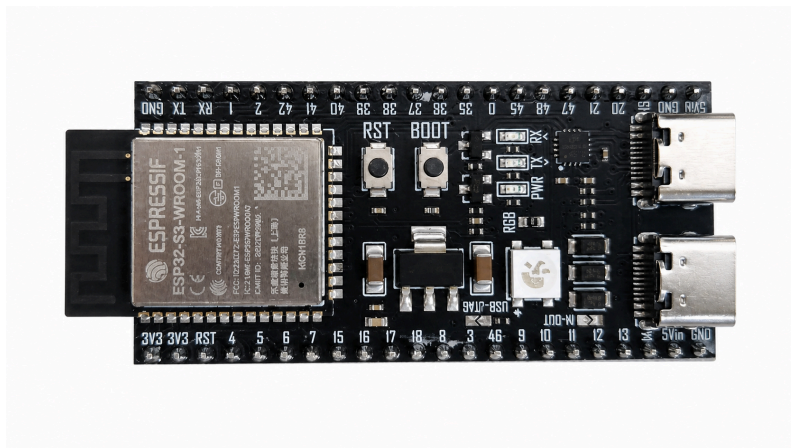
這支程式沒有持續洗出文字：沒有按鈕操作時，畫面不增加是正常的。看不到起始文字，先回第 33 頁排查，不靠接線嘗試解決上傳問題。

看到起始文字後，再關閉 Monitor、拔 USB，確認 PWR 熄滅。這個順序先把兩腳改為輸入，避免把按鈕接到舊程式的輸出腳。此時先照做，程式原理在做出數字後講。

下一頁：USB 保持拔除，放第二顆按鈕，先用電表確認方向。

原本的加鍵留下，再放一顆減鍵

先放按鈕；此頁仍不接開發板、不插 USB。



新增加的 GPIO5 是下排印字「5」，在「4」與「6」之間。不是從旁邊數第五腳。圖示板型不同時，先停止並核對。

哪顆按鈕	四腳放在哪裡	稍後接哪個 GPIO
原本那顆，貼「+」	e27、f27、e29、f29	GPIO4
新增加的，貼「-」	e21、f21、e23、f23	GPIO5

減鍵也要測，不用外形猜方向

沿用第 2–4 頁的電表通斷檔，以及公對公線引出測點。兩顆按鈕現在都沒有接開發板。

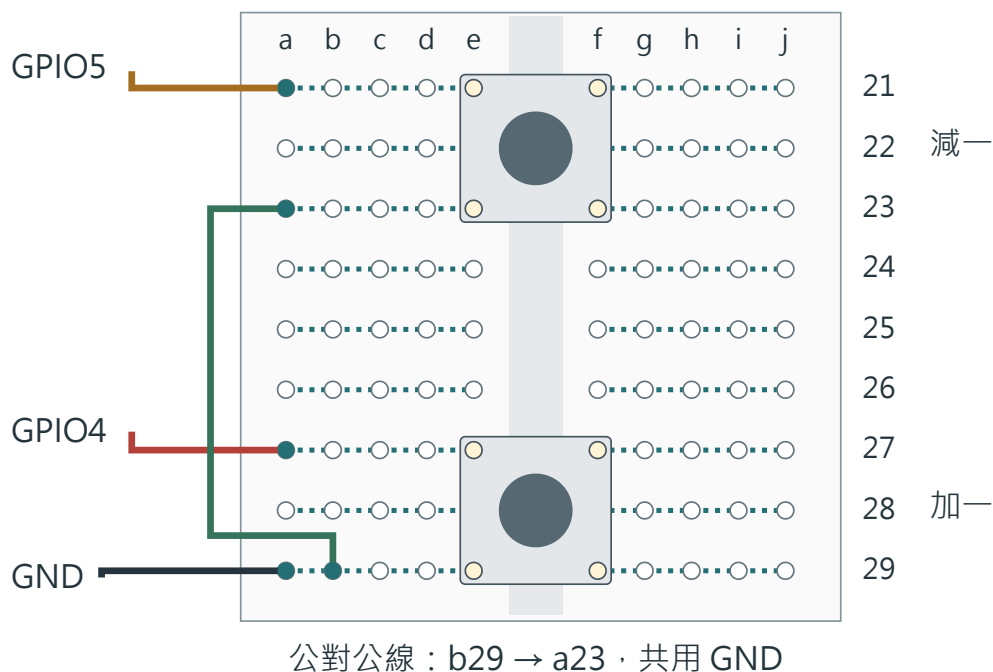
減鍵測點	預期
a21 與 j21，放開	會叫，同一組腳
a23 與 j23，放開	會叫，另一組腳
a21 與 a23，放開 → 按住	不叫 → 會叫

測試不符就停在方向與接觸檢查，不通電試錯。完成後移除量測線，電表 OFF、表筆放旁邊。

下一頁：先恢復加鍵兩條線，再增加減鍵兩條線。

四條線，連好加鍵與減鍵

USB 仍拔除。兩顆按鈕共用同一個 GND。



俯視關係圖，圓點表示線插入的孔；線段交叉但沒有圓點，不表示相接。板卡放桌上，不插在麵包板；以孔位與接點辨認，不靠線色。

順序	線材	兩端接點
1. 恢復加鍵	公對母	GPIO4 → a27
2. 恢復 GND	公對母	第 5 頁指定 GND → a29
3. 增加減鍵	公對母	GPIO5 → a21
4. 讓減鍵也接到 GND	公對公	b29 → a23

為什麼用 b29？a29 已插 GND 線，b29 和 a29 是板內相通的孔。從 b29 拉線到 a23，就是把 GND 接到第二顆按鈕的另一組腳。

全班一起核對四條線：不接 3V3、5Vin、TPO 或其他模組。GPIO4 與 GPIO5 不可接在同一列相通的孔。電表、量測線已移除，才進下一頁。

下一頁：插回 USB，看數字能否依照手的動作加減。

加、加、加，再減、減

先不用解釋程式。用自己的操作確認這個計數器。

1. 插回 **COM USB** 接頭，開 Serial Monitor，選 **115200**。
2. 兩鍵都放開，短按板上 RST，等一秒。看見 **event=start count=0** 後開始。
3. 每次按約半秒、放開約半秒，再按下一次。不要重疊按兩鍵。

依序操作	應看到的數字	我實際看到
第一次按加鍵	1	_____
第二次按加鍵	2	_____
第三次按加鍵	3	_____
第一次按減鍵	2	_____
第二次按減鍵	1	_____

預期訊息，不是本次實測

```
event=start count=0
event=plus count=1
event=plus count=2
event=plus count=3
event=minus count=2
event=minus count=1
```

plus 是加一，**minus** 是減一，**count** 後面是計算後的數字。這次只在開始或接受操作時留一筆紀錄，不每隔 0.3 秒重複印出狀態。

接著按住加鍵兩秒。預期只從 1 變 2，不會一直增加。放開後，再按一下才變 3。

下一頁：用剛剛的結果，說明數字存在哪裡、按住為什麼不連加。

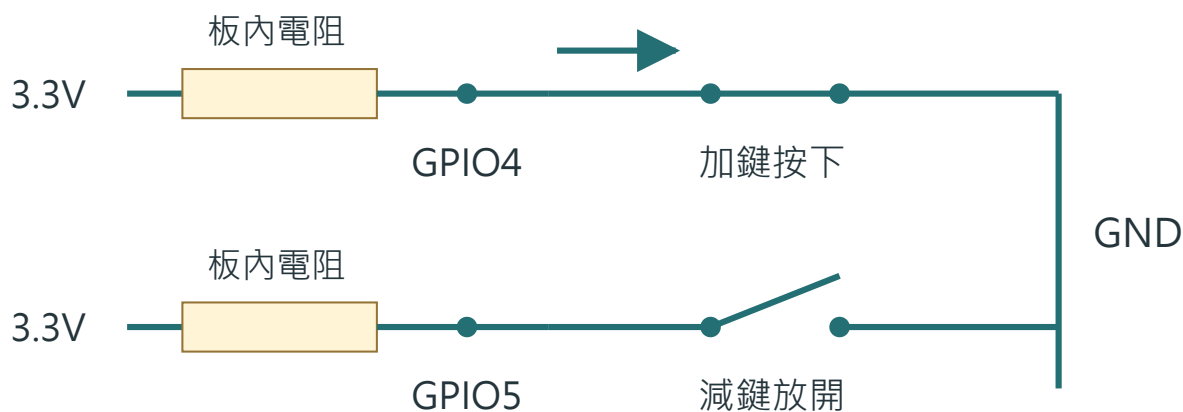
為什麼按一次，只加一次？

電線只告訴板子按鈕狀態；「加一」是程式訂的規則。

數字存在 ESP32 執行中的程式裡

`count` 是存放目前數字的變數。假設原本是 2，`count = count + 1`；的意思是「先算 2 + 1，再把 3 存回 `count`」。不是數學上永遠成立的等式。

`count = count - 1`；就是把目前數字減一再存回。電腦只接收並顯示結果；本版沒有把 `count` 存進斷電後仍保留的儲存空間，所以重啟會歸零。



電流路徑示例：只有加鍵按下。上方有經電阻限流的小電流回到 GND；下方按鈕開路。兩腳各有自己的板內上拉電阻，GPIO 只是讀取接點狀態。

避免多算，其實是兩件事

可能發生什麼	程式怎麼處理
手按住兩秒，程式讀到很多次「按下」	接受一次後暫停計數；兩鍵都放開並穩定後，才準備接受下一次。
按鈕剛接觸時，短時間反覆接通、斷開	讀值連續 40 毫秒沒變才接受，這是本例的去抖設定。

40 毫秒 = 0.04 秒，是本範例的選擇，不保證適用所有按鈕。太短的按下或放開可能不被接受；因此先用半秒的動作練習。兩鍵共用穩定等待時間，其中一鍵變化也會重新等待。

下一頁：不是只有「能加減」就完成，還要試長按、上下限與重新啟動。

把計數器的規則逐項試出來

先寫預測，再操作；觀察不符就保留紀錄，不改成標準答案。

測試條件與動作	預期	實際結果
RST 後，放開兩鍵一秒，再按減鍵	minimum，維持 0	_____
從 0 按加鍵並保持兩秒	只加到 1	_____
放開半秒，再按加鍵	加到 2	_____
放開後重做「加、加、減」三輪	從 2 依序到 3、4、5	_____
數字不是 0 時按 RST	start，回到 0	_____
啟動時按住加鍵，再放開；之後再按	啟動不計數，重新按才加到 1	_____

不用按 99 次，也能測上限

拔 USB、拔下板上所有杜邦線，再只接 USB。把 `MAX_COUNT = 99` 改成 `MAX_COUNT = 3` 並上傳。拔 USB 後照第 25 頁恢復四線，再接 USB、開 Monitor 選 115200。放開兩鍵、按 RST，從 0 逐次加；第四次應為 **maximum count=3**。這是測上限規則，不是實測到 99。

兩鍵一起按，不能拿來加減相抵

程式讀到兩鍵都按下，就等待兩鍵都放開。若第一顆早已被接受，原本那一次不會撤銷；本例不是精密的同時按壓辨識器。請測試「一起放開後，下一次單鍵能否正常計數」。

想一想：重新開啟電腦的 Monitor，一定會讓 count 歸零嗎？不一定；只有 ESP32 真的重新啟動才會。看有沒有新的 `event=start`，不能只看電腦視窗開關。

保留作品證據：一張接線照片、一段加減紀錄，以及本頁實際結果。記得寫下測試用的 `MAX_COUNT`。

作品先到這裡。結束時關閉 Monitor、拔 USB；後兩頁是完整程式，供開檔或回看使用。

程式先記住哪些事？

先完成操作再回來看。前後兩頁屬於同一個 Arduino 草稿。

```
// YD-ESP32-S3 Type-A V1.5 only; disconnect external wiring before upload.
const int PLUS_PIN = 4;
const int MINUS_PIN = 5;
const int MAX_COUNT = 99;
const unsigned long DEBOUNCE_MS = 40;

int count = 0;
int lastPlus = HIGH;
int lastMinus = HIGH;
unsigned long changedAt = 0;
bool readyForPress = false;

void printCount(const char* event) {
  Serial.print("event=");
  Serial.print(event);
  Serial.print(" count=");
  Serial.println(count);
}

void setup() {
  pinMode(PLUS_PIN, INPUT_PULLUP);
  pinMode(MINUS_PIN, INPUT_PULLUP);
  Serial.begin(115200);
  changedAt = millis();
  printCount("start");
}
```

程式內容	對應剛才的操作
PLUS_PIN、MINUS_PIN	加鍵接 4，減鍵接 5
count、MAX_COUNT	目前數字與上限；最低是 0
readyForPress	是否已放開兩鍵，準備接受下一次
setup()	重新啟動時設定兩腳輸入，開始傳訊息

下一頁：把 loop() 接在同一個草稿後面。不要只上傳這一頁。

反覆查看，必要時才改數字

此段接在上一頁後面；不要再貼一份 setup()。

```
void loop() {
  int plus = digitalRead(PLUS_PIN);
  int minus = digitalRead(MINUS_PIN);
  unsigned long now = millis();

  if (plus != lastPlus || minus != lastMinus) {
    lastPlus = plus;
    lastMinus = minus;
    changedAt = now;
  }
  // Overlapping presses require both buttons to be released again.
  if (plus == LOW && minus == LOW) readyForPress = false;
  if (now - changedAt < DEBOUNCE_MS) return;

  if (plus == HIGH && minus == HIGH) {
    readyForPress = true;
    return;
  }
  if (!readyForPress) return;
  readyForPress = false;

  if (plus == LOW) {
    if (count < MAX_COUNT) {
      count = count + 1;
      printCount("plus");
    } else {
      printCount("maximum");
    }
  } else {
    if (count > 0) {
      count = count - 1;
      printCount("minus");
    } else {
      printCount("minimum");
    }
  }
}
```

官方原理依據：[Arduino State Change Detection](#)、[Arduino Debounce](#)、[Espressif GPIO](#)。本例另訂兩鍵需放開、0~99 上下限規則；不能照搬官方 UNO 範例的 5V 接線。

本計數器已做軟體檢查；雙按鈕新接法仍須實物核對與上機驗證。範例畫面不是本次硬體測試結果。

最多五人的小房間

把剛才的計數器改成人數紀錄；沿用原接線，不新增零件。

情境：房間最多容納 5 人。加鍵模擬一人進入，減鍵模擬一人離開。這是手動計數，不會偵測真人或控制門鎖。

請修改原計數器，完成兩件事

1. 人數只能在 0~5 之間。0 人時按減鍵仍是 0；5 人時按加鍵仍是 5。
2. 保留原本每筆 `event=... count=...` 紀錄。每次印完紀錄，如果目前是 5 人，下一行再印 **FULL**，表示已滿。不到 5 人時，不新增這行。

保留長按一次、兩鍵放開才接受下次、RST 歸零的規則。**不改 GPIO 或接線**。可參考既有範例與講義，先自己修改再翻頁核對。

先填預測，再測試：前四列依序操作，每次按半秒、放開半秒。最後一列重新按 RST 後再測。

操作	預測人數 / 有無新 FULL	實際結果
RST，兩鍵放開一秒；按減鍵	_____ / _____	_____
依序按加鍵五次	_____ / _____	_____
再按第六次加鍵	_____ / _____	_____
按減鍵一次	_____ / _____	_____
RST 後放開一秒，長按加鍵兩秒	_____ / _____	_____

再回答：已經顯示過 FULL，減成 4 人後，舊的 FULL 還留在視窗中，代表現在仍然滿了嗎？應該看哪筆紀錄？

上傳修改版前，沿用第 23 頁：斷電拆開發板外接線，再只接 USB 上傳。完成後再斷電，依第 25 頁恢復四條線。不要帶電改線。

完成程式、預測與實測紀錄後再翻頁。下一頁就是參考解答。

上限改成五，滿了再印 FULL

只需修改上限與印出紀錄的函式，其餘計數與接線不變。

修改一：把原本那行換成 `const int MAX_COUNT = 5;`，不要另加第二個同名設定。

修改二：用下列內容取代原本整個 `printCount()`：

```
void printCount(const char* event) {
    Serial.print("event=");
    Serial.print(event);
    Serial.print(" count=");
    Serial.println(count);
    if (count == MAX_COUNT) Serial.println("FULL");
}
```

`==` 是比較兩邊是否相等，不會修改 `count`；`=` 是把右邊的數值存回左邊。兩者用途不同。

上一頁的測試	答案：數字與新訊息
0 人時按減鍵	0 ; minimum，不新增 FULL
加鍵五次	1、2、3、4、5；第五次新增 FULL
第六次加鍵	維持 5；maximum，仍新增 FULL
減鍵一次	4；minus，不新增 FULL
重新 RST 後長按加鍵	只到 1；不新增 FULL

第五次加鍵、第六次加鍵、減鍵的預期紀錄

```
event=plus count=5
FULL
event=maximum count=5
FULL
event=minus count=4
```

觀念題：舊 FULL 是過去的紀錄，不是現在的狀態。看最新的 **count=4**，現在沒有滿。程式不會自動刪除舊訊息。

以上為參考答案與程式預期輸出，不是實機結果。完整解答另附 `counter_exercise_solution.ino`；仍需照題目自行觀察與留下紀錄。

停在哪一步，就處理那一步

不要同時換線、換板型、換程式。一次只改一件事，才知道問題在哪裡。

現在遇到什麼	先一起做這些
不能安裝 IDE	確認下載的是 Windows 64-bit .exe。學校電腦要求管理權限時，交由管理人員處理，不繞過限制。
找不到 esp32 / 安裝失敗	回第 7 頁核對網址與網路；第 8 頁選 Espressif Systems。保存錯誤文字，不反覆亂換版本。
沒有新增 COM	板子先不接杜邦線。換已確認可傳資料的 USB 線，插板背 COM 接頭、直接接電腦。比較拔掉 / 插回前後清單。
有供電，仍沒有 COM	在 Windows 開始鈕按右鍵 → 裝置管理員，看「連接埠 (COM 和 LPT)」或未知裝置。圖示板使用 CH343；需要驅動時只用 WCH 官方 CH343SER，不隨便裝別型號。
上傳失敗	先關閉所有 Serial Monitor，核對第 10–11 頁的板型 / Port / 選項。等待其他程式釋放連接埠，再重試一次並保存錯誤文字。
卡在 Connecting，無法連上	僅對圖示板，無外接線時：重試 Upload，在 Connecting 時按住 BOOT、點一下 RST；開始 Writing 後放開 BOOT。上傳完成後必要時點一下 RST 啟動。這不是每次必做。
沒有 Hello / 是亂碼	確認正在看 Serial Monitor 而非 Output；速度選 115200。確認已上傳新程式，而不是只按勾勾編譯；USB 接頭是 COM。

改實體接線前拔 USB、移除其他電源。板子異常發熱、有異味或冒煙，立即停止供電，不繼續測試。

驅動程式官方來源：[WCH CH343SER](#)。無權限或裝置身分不明時，先由教師與管理人員一起確認，不盲目安裝。

操作依據與圖片來源

這份包含從零操作、按鈕紀錄與計數器；不是整堂 Week 2 的全部內容。

Arduino 官方文件與截圖

- [Downloading and installing the Arduino IDE 2](#)：安裝總覽，作者 Karl Söderby。
- [How to upload a sketch with the Arduino IDE 2](#)：選單、上傳按鈕，作者 Karl Söderby、Jacob Hylén。
- [Using the Serial Monitor tool](#)：訊息視窗，作者 Karl Söderby。
- [Select board and port in Arduino IDE](#)：板型與連接埠的操作。

Arduino 截圖取自官方 docs-content，未修改影像；原圖採 [CC BY-SA 4.0](#) 授權。舊版截圖只用來辨認入口，不代表本課的板型、速度或版本。

Espressif 官方文件與截圖

- [Getting Started with ESP32 Arduino](#)，Jan Procházka，2025-10-01：偏好設定與管理員畫面，未修改影像；原圖權利屬 Espressif。
- [Installing](#)：穩定版安裝網址。
- [Arduino IDE Tools Menu](#)：設定意義。
- [GPIO](#)：按鈕程式與內部上拉。

實物與量測

板卡、電表與麵包板照片為本課既有實物照片。孔位圖是教學示意；不把圖片當成所有同名商品都相同的保證。通斷原則參考 [Fluke 通斷測試說明](#)。

所有程式輸出示例都是預期格式，不是這次操作的實測紀錄。學習者應以自己實際看到的畫面記錄結果。網路來源查核：2026-09-16。